Analytical Quantum Layout Synthesis

Yuchen Zhu*, Nikos Hardavellas
Department of Computer Science, Northwestern University
*yzhu7@u.northwestern.edu

Introduction & Problem Formulation

Goal: Minimize SWAP overhead in Quantum Layout Synthesis (QLS) to enable efficient quantum circuit execution on near-term hardware.

Challenge: QLS is NP-hard. Prior approaches fail to scale:

- **Exact methods**: Limited to ∼10 qubits; impractical for current devices.
- **Heuristics (SABRE)**: Optimality gap up to 63× in SWAPs; >46 min and 100 GB RAM for 200-qubit, 15K-layer circuits.

Hardware pressure: emerging devices are moving toward 1 ms qubit lifetimes and 500+-qubit wafer-scale chips. At ∼68 ns per two-qubit gate, compilers must reason across ∼15K executable layers before decoherence dominates.

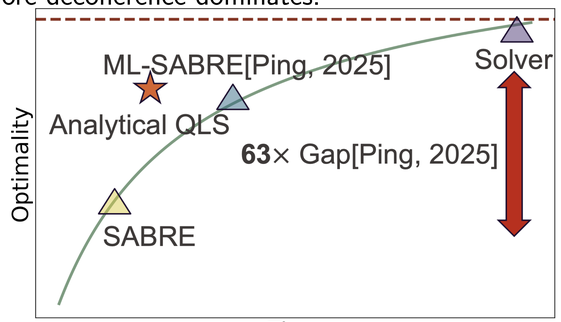

Figure: Analytical QLS reduces SWAPs while scaling to larger circuits.

Key Insight: Treat *per-layer mapping* as the first class citizen. Let D be the hardware distance matrix, A_l the gate interaction matrix at layer l, and P_l the qubit permutation.

Problem setting: input is a circuit and hardware topology; output is a legal mapped gate sequence; objective is to minimize inserted SWAPs.

Gate cost (QAP):

$$f_{\text{gate}}(P_l) = \sum_{i,j} \sum_{u,v} A_l[i,j] \cdot P_l[i,u] \cdot P_l[j,v] \cdot D[u,v]$$

Layer consistency penalty:

$$f_{\text{layer}}(P_l, P_{l-1}) = \sum_{i} \sum_{u,v} P_l[i,u] \cdot P_{l-1}[i,v] \cdot D[u,v]$$

Total cost (λ annealed upward over iterations):

$$f = \sum_{l} f_{\text{gate}}(P_l) + \lambda \sum_{l} f_{\text{layer}}(P_l, P_{l-1})$$

The first-layer mapping is optimized in the same formulation instead of being chosen randomly, reducing early placement debt.

Method: Analytical Optimization

Circuit Layering: Either *depth-preserving* (maximal parallel gate sets) or *distance-based* (gates with qubit distance $> k$ separated), trading parallelism for routing simplicity.

Sinkhorn Relaxation: P_l is binary and non-differentiable. Relax to a doubly stochastic matrix:

$$L = \frac{X}{\tau}, \quad P = \exp(f + g + L)$$

$f = -\text{LSE}(g+L)$, $g = -\text{LSE}(f+L)$; temperature τ is annealed to produce a hard assignment.

Nesterov Accelerated Gradient (NAG):

$$X_{t+1} = X_t + \mu(X_t - X_{t-1}) - \eta \nabla f(X_t + \mu(X_t - X_{t-1}))$$

After convergence, P_l is discretized via arg max and a greedy pass inserts SWAPs between layers.

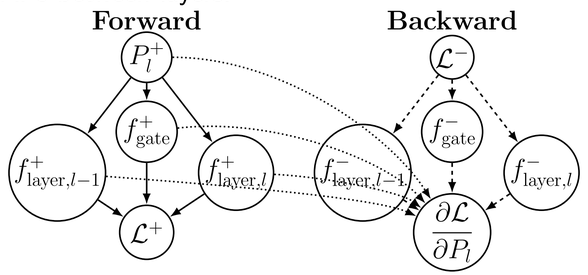

Figure: Forward/backward propagation mirrors neural network training.

Deep Learning Analogy: gate cost ↔ prediction error; layer cost ↔ regularization; Sinkhorn layer ↔ activation function. Enables auto-diff, GPU acceleration, and standard optimizers out of the box.

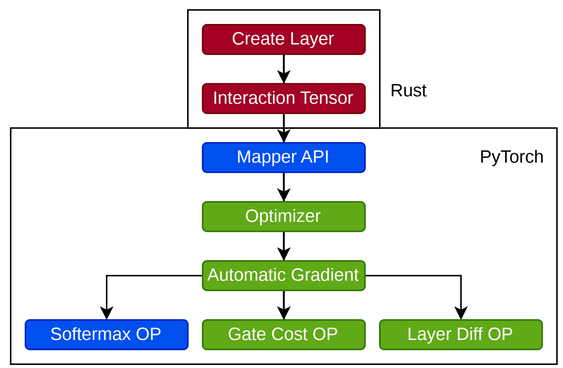

Figure: Software architecture: Rust layer construction plus PyTorch optimization.

Post-processing: soft placements are hardened by arg max, then a greedy pass resolves the largest remaining physical-distance mismatch first.

Evaluation

Setup: 54-qubit circuits, up to 1,500 layers, on a 64-qubit grid. Compared against exact methods and SABRE.

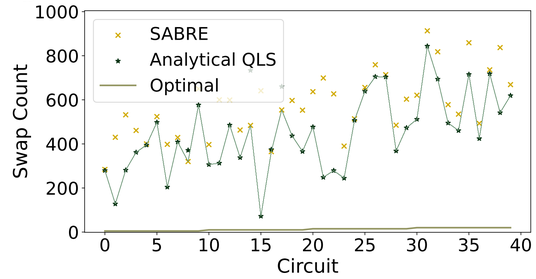

Figure: Analytical QLS outperforms baselines in 86% of circuits.

Key results:

- **86%** of circuits: fewer SWAPs than all prior methods
- **45%** average SWAP reduction; best case **1.1** times of optimal
- Near-optimal behavior on several known-optimal benchmarks

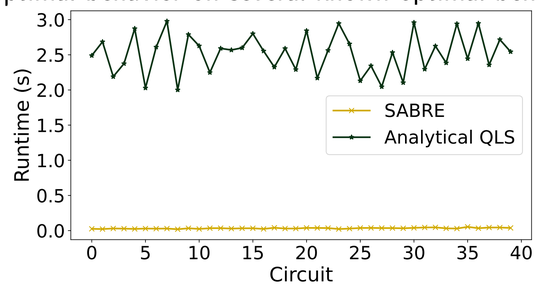

Figure: Runtime comparison: higher small-circuit overhead, stronger large-circuit scaling.

Scaling stress test: on 200-qubit, 15K-layer circuits, Analytical QLS finds a solution in 10 min / 16 GB with **80%** fewer SWAPs; SABRE requires 46+ min / 100 GB.

Why it helps: SABRE optimizes a local window, while Analytical QLS optimizes globally over per-layer placements and directly penalizes cross-layer inconsistency.

Conclusion & Future Work

Contributions: First gradient-based analytical QLS; near-optimal SWAPs at scale; bridges quantum compilation and deep learning.

Future Work:

- Optimal layering strategy selection
- Sparse interaction matrix for memory efficiency
- Extend cost to gate fidelity and coherence time
- Fault-tolerant regime mapping